

```
});
?>
```

Now, all we have to do is print these values using TWIG's *for* loop; so this is the final *page.html* file:

```
hi {{name}},
<br />
```

This is the list of the friends who registered:

```
<br />
{%for user in users%}
{{user.name}} who is {{user.age}} years old.<br />
{%endfor%}
```

You might notice here that the functional units in TWIG also supports conditional statements such as *if* and *else* in the same fashion as the *FOR* looping block we saw above.

Conditional statements: It is not surprising that TWIG also supports conditional statements such as *if* and *else* in the same fashion as the *FOR* looping block we saw above.

Including documents: This is a nice way to include other documents/plugins in your Web page (just like PHP does). `{% include 'pages/xyz.html' %}` will do the job neatly and cleanly!

Inheritance: This one's a beast; all the languages out there have made peace with this feature that is great in preventing code redundancy and spaghetti code. With inheritance applied in a templating engine, we can achieve the milestone of creating a master-template, which will act as a frame to fit in the pieces we want to. Here's a good example of this feature.

Create a file in the views folder named *base.html.twig* and write the following code into it:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
<title>
{% block title %}Welcome!{% endblock%}
</title>
{% block stylesheets %}{% endblock %}
<link rel="icon" type="image/x-icon" href="{{
asset('favicon.ico') }}" />
</head>
<body>
{% block body %}{% endblock %}
{ % block javascripts %}{% endblock %}
</body>
</html>
```

All we need to do is extend this template, and overwrite those blocks we created individually, like shown below:

IDEs integration

Many IDEs support syntax highlighting and auto-completion for TWIG. These are:

- *Textmate* via the TWIG bundle
- *Vim* via the Jinja syntax plugin or the vim-twig plugin
- *Netbeans* via the TWIG syntax plugin (until 7.1, native as of 7.2)
- *PHPStorm* (native as of 2.1)
- *Eclipse* via the TWIG plugin
- *Sublime Text* via the TWIG bundle
- *GtkSourceView* via the TWIG language definition (used by Gedit and other projects)
- *Coda* and *SubEthaEdit* via the TWIG syntax mode
- *Coda 2* via the other TWIG syntax mode
- *Komodo* and *Komodo Edit* via the TWIG highlight/syntax check mode
- *Notepad++* via the Notepad++ TWIG Highlighter
- *Emacs* via web-mode.el
- *Atom* via the PHP-TWIG for atom

```
{% extends './base.html.twig' %}
{% block body %}
  {% include 'plugins/carousel.html' %}
  this is the homepage!
{% endblock %}
{% block title %}HOME{% endblock %}
```

So here, what you can see is for every page. You just have to define what you want in each block; also, no specific order is required (but recommended) to edit these blocks. I can edit the title block after editing the body block, and it won't make any difference.

In this article, we have covered just four of the features that we use in our daily work as developers, but the list keeps growing for TWIG's features.

You can play with many other features while using the MVC and CMS architectures like building forms out of the objects, creating URLs from the route names, and many others. For more detailed documentation and features, you can visit: <http://twig.sensiolabs.org/documentation>. 

References

- [1] Book: The TWIG book (offline documentation)
- [2] Website: twig.sensiolabs.org, <http://twig.sensiolabs.org/documentation>
- [3] Tutorials: KNP University tutorials and the YouTube channel [codecourse](https://www.youtube.com/channel/UCcodecourse)

By: Prakash Patel and Sameer Manek

Prof. Prakash Patel works as an assistant professor in the computer engineering department at Nirma University. He can be contacted at prakash.patel@nirmauni.ac.in.

Sameer Manek is in the final year of his diploma in computer engineering at Nirma University. He can be contacted at maneksameer1@gmail.com